



LangChain – Structured Output

By 

LangChain - output

- ❖ **Structured Output** → LLM is **FORCED** to output your schema.
- ❖ **Output Parsers** → You **FIX** the LLM output **AFTER** it is generated.

Structured Output (LLM produces perfect structured JSON/Pydantic)

- ✓ You tell the model the exact schema
- ✓ Model **MUST** output that schema
- ✓ No need for parsing
- ✓ Most reliable and recommended

This happens *before* generation.

The model is constrained at generation time.

2. Output Parsers (LLM outputs text → You parse it later)

- ✓ LLM writes freely
- ✓ You try to extract structure after the fact
- ✗ Not guaranteed to follow schema
- ✗ More error-prone

This happens *after* generation.

You clean/process the text.

When to use

When to Use What (Rule of Thumb)

✓ Use Structured Output when:

- ❖ You need **fixed JSON schema**
- ❖ You are building **RAG pipelines**
- ❖ You are building **agents & tool calls**
- ❖ You need **reliable API-like responses**
- ❖ You want **zero parsing errors**

✓ Use Output Parsers when:

- ❖ LLM output can be **free-flowing**
- ❖ You only want **small extraction**
- ❖ You want flexibility over strictness
- ❖ You work with **unstructured text**



With_structured_output - TypedDict

```
from langchain_google_genai import ChatGoogleGenerativeAI
```

```
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
```

```
class Person(TypedDict):
```

```
    name: str
```

```
    age: int
```

```
    DOB: str
```

```
structured_model = model.with_structured_output(Person)
```

```
result = structured_model.invoke("From this provide name of the player and age. Context :  
Sachin Tendulkar is 52 years old, as of November 2025. He was born on April 24, 1973.")
```

TypeDict - Annotated

from typing import TypedDict, Annotated, Optional, Literal **###** Annotated comes from **typing.Annotated**.

“**Annotated**” attach extra information (metadata) to a type — typically used by frameworks like LangChain to provide descriptions.

Annotated[actual_type, metadata]

Ex: summary: **Annotated**[str, "A brief summary of the review"]

Actual_type can be str or Optional or List[str] etc.

Optional[T]:

The value can be **T** or **None**.

Ex: name: **Annotated**[Optional[str], "Write the name of the reviewer"]

 strenth: **Annotated**[Optional[list[str]], "Write down all the strenth inside a list"]

Behind the scenes, LangChain builds something like:

```
{
  "summary": {
    "type": "string",
    "description": "A brief summary of the review"
  }
}
```

Part	Purpose
list[str]	Actual type
Literal["pos","neg"]	Restrict values
Optional[str]	Allow None
Annotated[..., "description"]	Add metadata for LLM, or instruction for the field

- Keep schema + instructions in one place
- Make structured output more accurate
- Avoid writing long prompt instructions separately
- Create self-documenting schemas

Pydantic - validation library



- ❖ **TypedDict** - structure definition only, no validation.
- ❖ **Pydantic** - A powerful Python library for defining **data models with validation**, enforces types **at runtime**.
- ❖ Pydantic provides default values, coerce(auto type conversion) and Field can be used to provide description, constraint, default values, regex expressions

```
from langchain_google_genai import ChatGoogleGenerativeAI
from pydantic import BaseModel, EmailStr, Field
from typing import Optional
```

```
class Student(BaseModel):
```

```
    name: str = 'Raju'
```

```
    age: Optional[int] = None
```

```
    email: EmailStr
```

```
    cgpa: float = Field(gt=0, lt=10, default=5, description='A decimal value representing the cgpa of the student')
```

```
new_student = {'age': '32', 'email': 'abc@yahoo.com'} # age will auto to int i.e. Coercion means automatic type conversion
```

```
student = Student(**new_student)
```

```
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
```

```
structured = model.with_structured_output(Student)
```

```
result = structured.invoke("Give details of Mahatma Gandhi")
print(result)
```

Pydantic - Field

- ❖ Field can be used to provide description, constraint, default values, regex expressions
- ❖ Ex: `cgpa: float = Field(gt=0, lt=10, default=5, description='A decimal value representing the cgpa of the student')`

```
from pydantic import BaseModel, Optional, Field
```

```
class PlaceInsights(BaseModel):
```

```
    place_name: str = Field(min_length=3, max_length=50, description="Name of the place")
    rating: Optional[float] = Field(default=None, ge=0, le=5, description="Overall rating out of 5")
    pincode: Optional[str] = Field(default=None, pattern=r"^\d{6}$", description="6 digit postal code")
```

```
    # Custom validation
```

```
    @field_validator("rating")
```

```
    def validate_rating(cls, value):
```

```
        if value is not None and value < 1:
```

```
            raise ValueError("Rating should not be extremely low")
```

```
        return value
```

LLM will generate the JSON text and LangChain will convert the JSON text to python dict object if we use TypedDict and pydantic object if Pydantic is used.

Ex: `Print(type(result))`

JSON Schema

```
json_schema = {  
    "title": "MovieFeedback",  
    "type": "object",  
    "properties": {  
        "movie": {  
            "type": "string",  
            "minLength": 2, "maxLength": 100  
        },  
        "sentiment": {  
            "type": "string",  
            "enum": ["pos", "neg", "neutral"]  
        },  
        "likes": {  
            "type": "array",  
            "items": {"type": "string"},  
            "minItems": 1, "maxItems": 5  
        },  
        "dislikes": {  
            "type": "array",  
            "items": {"type": "string"},  
            "maxItems": 5  
        },  
        "reviewer": {  
            "type": "string",  
            "pattern": "^[A-Za-z ]+$"  
        }  
    },  
    "required": ["movie", "sentiment"],  
    "additionalProperties": False  
}  
structured_model = model.with_structured_output(json_schema)
```

TypedDict Vs Pydantic Vs JSON Schema

TypedDict

You only need **type hints**, not real validation.

You **don't require validation** like checking allowed values or constraints.

You **trust the LLM** to give correct structured output.

Pydantic if:

- You need **runtime data validation** (e.g., allowed values, types, constraints).
- You want **default values** when the LLM misses fields.
- You want **automatic type conversion** (e.g., "100" → 100).

JSON Schema if:

You don't want to import extra Python libraries (like Pydantic).

You need **validation**, but you don't need Python objects.

You want structure defined in **standard JSON format** (OpenAPI-style).

Feature	TypedDict	Pydantic	JSON Schema
Runtime validation	✗ No	✓ Yes	✓ Yes (via jsonschema)
Default values	✗ No	✓ Yes	✗ Not by itself
Type conversion	✗ No	✓ Yes	✗ No
Requires Python class	✓ Yes	✓ Yes	✗ No
Standard outside Python	✗ No	✗ No	✓ Yes
Best when	Fast & simple	Strong validation	Multi-language, strict schemas

Thank you